Real-world (act out a canary release) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `canary_release_practice.py`: a canary release sends only a small share of real traffic to the new version and measures it directly against the old one, before the new version ever reaches everyone. Actually simulating this on your own small project turns it from a buzzword into something you can explain with real numbers. **If not:** "we'd do a canary release" would stay a phrase you've heard, not something you can back up with actual numbers.

All examples below use this shared setup — 20 fake test requests, and a `run_eval_suite` stub where `"v2"`'s prompt scores better than `"v1"`'s:

```
# canary_release_practice.py
import random

prompts = {
    "v1": "Answer the customer's question.",
    "v2": "Answer the customer's question politely and cite the source.",
}
test_requests = list(range(20))  # 20 stand-in requests

def run_eval_suite(prompt_text):
    if "politely" in prompt_text:
        return {"score": 1.0}
    return {"score": 0.6}
```

Each example below calls `random.seed(...)` with a fixed number so the "random" routing is the same, reproducible split every time you run it — remove the seed line for genuinely random routing in real use.

Solution

Story: quality score isn't the only number that genuinely matters — a marginally better answer that costs twice as much to produce may not actually be worth shipping. And an empty canary group, with zero samples, should say explicitly *why* it's empty rather than silently producing a meaningless number. **If not:** you'd promote purely on score alone, and a small sample run would crash outright with a bare, unexplained `ZeroDivisionError`.

```
# canary_release_practice.py
import logging

logger = logging.getLogger(__name__)
logger.setLevel(logging.DEBUG)
console = logging.StreamHandler()
console.setLevel(logging.INFO)
logfile = logging.FileHandler("canary.log")
logfile.setLevel(logging.DEBUG)
logger.addHandler(console)
logger.addHandler(logfile)

class EmptyGroupError(Exception):
    pass

random.seed(1)

def route_and_score(
    test_requests: list, prompts: dict, canary_share: float = 0.2
) -> list[dict]:
    results = []
    for request in test_requests:
        # how: random.random() is between 0 and 1, so this is true
        # for about canary_share of all requests
        if random.random() < canary_share:
            version = "v2"
        else:
            version = "v1"
        logger.debug(f"request {request} -> {version}")
        eval_result = run_eval_suite(prompts[version])
        results.append({
            "version": version,
            "score": eval_result["score"],
            "cost": len(prompts[version]) * 0.0001,
        })
    logger.info(
        f"Routed {len(results)} requests with canary_share={canary_share}"
    )
    return results

def summarize(results: list[dict], version: str) -> dict:
    matching = []
    for r in results:
        if r["version"] == version:
            matching.append(r)
    if not matching:
        # why: an empty group is a rollout problem, not a math bug —
```

```
        # say so, instead of crashing on division by zero
        raise EmptyGroupError(
            f"No requests were routed to {version} — "
            "send more requests or raise canary_share"
        )

    scores = []
    costs = []
    for r in matching:
        scores.append(r["score"])
        costs.append(r["cost"])

    return {
        "version": version,
        "count": len(matching),
        "avg_score": sum(scores) / len(scores),
        "avg_cost": sum(costs) / len(costs),
    }

results = route_and_score(test_requests, prompts)
control = summarize(results, "v1")
canary = summarize(results, "v2")
print(control)
print(canary)
```

Expected output:

```
Routed 20 requests with canary_share=0.2
{'version': 'v1', 'count': 15, 'avg_score': 0.6, 'avg_cost': 0.0031}
{'version': 'v2', 'count': 5, 'avg_score': 1.0, 'avg_cost': 0.006}
```

(the real printed float carries extra trailing digits from binary floating point, like `0.0030999999999999995` — rounded here for readability)

The terminal shows only the `INFO` line; `canary.log` *also* holds one `DEBUG` line per request, like `request 0 -> v2` and `request 1 -> v1`. With `random.seed(0)`, `summarize(results, "v2")` now stops with a clear `EmptyGroupError` ("No requests were routed to v2 — send more requests or raise canary_share") instead of a bare `ZeroDivisionError`.

Revision from Doc01 (Intermediate): two habits come back. First, Doc01's "logger with two output levels" exercise: the terminal gets one `INFO` summary, while `canary.log` keeps a `DEBUG` line for every single routing decision. When a customer says "I got the new answer style yesterday", the file tells you which version served each request — 20 lines in the terminal would just be noise. Second, a named error with a clear message: an empty group is not a bug in the math, it's a problem with the

rollout (too few requests or too small a share), and `EmptyGroupError` says exactly that, where `ZeroDivisionError: division by zero` says nothing useful.